

令和6年度学士論文

OpenMP offloadingを用いた並列 プログラムの異種GPU上での 評価

21B30971 吉田 和真
yoshida.k.bp@m.titech.ac.jp

指導教員 遠藤 敏夫

東京科学大学 情報理工学院
数理・計算科学系

令和7年2月28日

概要

計算の高速化の手法として、プログラムの並列化が挙げられる。並列化とは CPU や GPU 内に存在する複数のコアなどを用いて、プログラムの処理を細かいタスクに分割し、並列に実行する手法である。代表的な CPU 並列化手法としては OpenMP が、GPU 並列化手法としては、CUDA, OpenACC, ROCm, OpenMP offloading などが挙げられる。本研究では OpenMP offloading という AMD GPU や NVIDIA GPU 上で動作可能な GPU 並列化手法について取り上げる。

目次

第 1 章	序論	5
1.1	本研究の背景	5
1.2	本研究の目的	6
1.3	本研究の成果	6
1.4	本論文の構成	7
第 2 章	背景	8
2.1	GPU アクセラレータ	8
2.2	CPU, GPU の並列化の手法	9
第 3 章	並列化プログラム	11
3.1	行列積演算	11
3.2	diffusion	14
3.3	hotspot プログラム	17
3.4	ブロッキング機能を取り除いた hotspot プログラム	19
第 4 章	実験と評価	21
4.1	実行環境	21
4.2	各プログラムの実験方法	25
4.3	OpenMP offloading を用いた性能	26
4.4	OpenMP と OpenACC との比較実験	30
4.5	性能ポータビリティ	34

第 5 章	関連研究	38
5.1	プログラミングフレームワーク	38
第 6 章	結論	39
6.1	まとめ	39
6.2	今後の課題	39
謝辞		41
参考文献		42

図目次

4.1	各 GPU による行列積演算の実行速度	26
4.2	各 GPU による diffusion の実行速度	27
4.3	行列積演算プログラムでのデータ転送速度	28
4.4	diffusion プログラムでの転送速度	29
4.5	マシン 1 OpenMP との行列積演算速度比較	30
4.6	マシン 2 OpenMP との行列積演算速度比較	31
4.7	マシン 2 OpenACC との行列積演算速度比較	31
4.8	マシン 3 OpenMP との行列積演算速度比較	31
4.9	マシン 3 OpenACC との行列積演算速度比較	31
4.10	マシン 1 における diffusion の実行速度	32
4.11	マシン 2 における diffusion の実行速度	32
4.12	マシン 1 における diffusion の実行速度	32
4.13	マシン 2 における diffusion の実行速度	33
4.14	マシン 3 における diffusion の実行速度	33
4.15	異なる for ループでの OpenMP 実行速度	34
4.16	異なる for ループでの OpenMP offloading 実行速度	35
4.17	異なる for ループでの OpenACC 実行速度	35
4.18	OpenMP での hotspot 実行速度	37
4.19	OpenMP offloading での hotspot 実行速度	37

表目次

4.1	マシン 1 の構成	22
4.2	マシン 2 の構成	23
4.3	マシン 3 構成	23
4.4	使用したコンパイラの一覧	24

第 1 章

序論

1.1 本研究の背景

近年, AI 技術が目まぐるしく進化している. 機械学習やディープラーニングの発展から始まり, ChatGPT や生成 AI など以前では考えられない技術の発明に至った. メモリやストレージ容量の増加など要因はさまざまであるが, 中でも, 計算の高速化はこの目覚ましい技術の発展に大きく貢献している. 計算の高速化技術は GPGPU[1] という考え方の登場により, 飛躍的に進歩することとなった. GPGPU とは, 本来グラフィック処理を目的として広く使用されてきた GPU を並列計算に応用するという考え方である. 1997 年に OpenMP Architecture Review Board(ARB) という非営利団体によって, 並列計算を目的とした API である OpenMP[2] が開発されたように, 以前から CPU での並列計算は行われてきたが, GPU では行われてこなかった. しかし, 2007 年 NVIDIA 社が GPGPU 開発環境である CUDA を公開すると, GPU での並列計算も可能となり, その計算の速さから世界中で GPGPU への関心と GPU の重要性が高まっていった. そうして, GPGPU の考え方を元に CUDA 以外にも OpenACC[3], OpenCL, ROCm, OpenMP offloading[4] など, 多くの GPU 並列化手法が生まれることとなった. こうして, 並列化は多種多様な方法で行われることとなった.

しかし, それぞれの並列化手法にも, メリット, デメリットが存在する. 例えば, CUDA はディープラーニングでも広く用いられており, GPU の並列化手法として最も有名であるが, 比較的程序が難しく, 学習コストが高い. また, 並列化を行う場合, 元のプログラムから大きく改変しなければいけない. しかし, OpenACC や OpenMP offloading

などは元のプログラムに指示文を付け加えるだけで並列化可能であるというメリットがある。さらに, OpenACC は NVIDIA GPU のみ使用可能で, CUDA は ROCm での変換ツールを用いれば NVIDIA GPU に加え, AMD GPU でも使用できる。一方, OpenMP offloading は変換ツールを用いることなく, NVIDIA GPU, AMD GPU 上で動作可能である。このように, 適用可能な GPU の種類も個数もそれぞれの並列化手法によって異なる。

本研究では, NVIDIA GPU と AMD GPU 上で動作可能であり, OpenMP にオフロード機能を搭載した OpenMP offloading という並列化手法を取り上げる。

1.2 本研究の目的

本研究の目的は, OpenMP offloading を用いた GPU 並列プログラムを AMD GPU と NVIDIA GPU 上で実行し, 異種 GPU 上での性能を評価することである。また, OpenMP offloading を OpenMP や OpenACC を用いたプログラムと比較して, CPU と GPU での性能の違いや, GPU 並列化手法としての評価を行う。さらに CPU と GPU における性能ポータビリティについて実験を行った。

1.3 本研究の成果

OpenMP offloading を用いて AMD GPU と NVIDIA GPU 上でプログラムを実行した際, AMD GPU 上で実行速度に大きな低下が見られることがあった。反対に NVIDIA GPU では CPU, GPU 間のデータ転送速度が不安定になる場合が確認された。OpenMP と OpenACC を用いた比較実験では CPU 並列化手法より GPU 並列化手法の方が全体的に実行速度が速い結果となった。また, OpenMP offloading は OpenACC と同等あるいはそれ以下の実行速度を出すことが確認された。さらに, CPU 並列化と GPU 並列化で性能ポータビリティが保たれていないケースが存在することが分かった。

1.4 本論文の構成

第 2 章では背景として本研究で用いる並列化手法についてそれぞれ解説し、第 3 章ではそれぞれの並列化手法に沿って作成した、行列積や物理学をモデルとしたプログラムについて説明する。第 4 章では OpenMP offloading で実装された、前章で紹介したプログラムを AMD GPU, NVIDIA GPU 上で実行・評価し、さらに OpenMP offloading について OpenMP と OpenACC を用いた場合の実験結果をもとに CPU と GPU での性能の違いや GPU 並列化手法としての評価も行う。加えて、CPU 実行と GPU 実行における性能ポータビリティに関する実験も行う。第 5 章では関連研究について紹介し、第 6 章ではまとめと今後の課題について述べる。

第 2 章

背景

本研究では異なる GPU アクセラレータ上における並列プログラムの評価実験を行う。本章では本研究で用いる GPU および並列化手法についてそれぞれ紹介する。

2.1 GPU アクセラレータ

演算性能を向上させる計算システムを実現するために、汎用 CPU に加えて、一部の演算を加速するアクセラレータを接続する手法が長く知られている。特に graphic processing unit (GPU) をアクセラレータとして用いることが世界のスーパーコンピュータや AI データセンターで常識となりつつある。GPU アクセラレータは主に NVIDIA, AMD, Intel が製造しているが、互いのソフトウェア互換性は低い状況にある。

2.1.1 NVIDIA GPU

NVIDIA 社は 1999 年に Geforce 256 という拡張ボードを公開し、これが世界初の GPU とされる。当時の GPU はコンピュータグラフィックス専用ハードウェアであったが、2007 年に汎用プログラミング可能な NVIDIA GPU 上とその上で利用可能な開発環境である CUDA を発表した。これにより大規模で汎用な並列計算が可能となり、東京工業大学 (当時)TSUBAME1.2 を含むスーパーコンピュータに採用されるようになった。その後も高い演算性能とメモリバンド幅を活用し、科学シミュレーションだけでなくディープラーニングや機械学習などの発展を促しており、さらに近年の生成 AI ブームに

より, さらに需要が高まっている.

NVIDIA GPU を用いたプログラミング手法としては, CUDA や後述の OpenACC, OpenMP offloading などが用いられる. OpenMP offloading が異種 GPU 間の互換性を保つために期待されているが, CUDA・OpenACC に比べ整備や普及が遅れている状況である.

2.1.2 AMD GPU

AMD GPU は GPU のブランド Radeon を確立した ATI technologies 社に起源を持つ. 2006 年 AMD 社が ATI technologies を買収すると, AMD Radeon として改良を進めてきた. グラフィックス以外の汎用計算の対応については NVIDIA に比較すると遅れている状況にあったが, 近年は開発環境 ROCm および OpenMP offloading によるプログラミングが可能となっている. 2024 年 11 月時点の Top500 スーパーコンピュータランキングにおいては, 世界一位の El Capitan と二位の Frontier が AMD GPU を採用している [5].

2.2 CPU, GPU の並列化の手法

以下では CPU や GPU の並列プログラミング環境として, 本研究で利用するものを紹介する.

2.2.1 OpenMP

OpenMP(Open Multi-Processing) とは共有メモリ型計算機で並列プログラミングを可能にする API(Application Programming Interface) である [2]. ソース上に指示文を挿入するディレクティブ・ベースという形態を取ることで, 既存のコードに指示文を挿入するだけで記述可能である. FORTRAN や C/C++ 言語に対応しており, マルチコア CPU で利用可能である.

2.2.2 OpenMP offloading

OpenMP offloading とは OpenMP にオフロード機能を加えた API である [4]. オフロード機能とは, 一部の処理を他のアクセラレータにオフロードして, 実行する機能のことであり, 本研究ではアクセラレータとして GPU を使用する. FORTRAN や C/C++ 言語に対応しており, AMD GPU, NVIDIA GPU など利用可能である. OpenMP でのディレクティブがそのまま使えるため, 学習コストが低い.

2.2.3 OpenACC

OpenACC とは OpenMP offloading と同じく, ディレクティブ・ベースであり, オフロード機能を備えた API である [3]. OpenMP offloading と大きく異なるのは, OpenMP とは異なるディレクティブを使用する点である. FORTRAN や C/C++ 言語に対応しており, 現時点では NVIDIA GPU のみで利用可能である.

第 3 章

並列化プログラム

本章では, 本研究で性能比較の対象とする並列プログラムとして, 行列積演算, diffusion, Rodinia ベンチマーク集の hotspot プログラムを紹介する. これらはディレクティブベースの開発環境である OpenMP, OpenMP offloading, OpenACC で記述されており, ディレクティブ部分以外は同一のプログラムコードを用いて NVIDIA GPU, AMD GPU, マルチコア CPU での性能評価を行う. さらに一部のプログラムについてはディレクティブ部分以外の改造も行っており, 性能におよぼす影響を 4 章で評価する.

3.1 行列積演算

行列積演算のプログラムを紹介する. このプログラムでは $m \times k$ の行列 A と $k \times n$ の行列 B の積を $m \times n$ 行列 C として求めている. なお, OpenMP では三重ループの順番が j, l, i であるのに対し, OpenMP offloading, OpenACC では j, i, l の順番であるのは, それぞれの並列化手法で実験した結果, 特に行列サイズが大きい場合において最適であると判明した為である. 詳細は第 4 章第 5 節「性能ポータビリティ」の箇所で説明する.

3.1.1 OpenMP を用いた行列積演算

OpenMP を用いた行列積演算をソースコード 3.1 に示す. なお, `#pragma omp for` の行で `collapse(2)` をディレクティブとして追加しないのは, 実験した結果, 計算速度が遅くなってしまった為である.

ソースコード 3.1 OpenMP を用いた行列積演算プログラム

```
1 int matmul()
2 {
3     int i, j, l;
4     #pragma omp parallel private(i,l)
5     #pragma omp for
6     for (j = 0; j < n; j++) {
7         for (l = 0; l < k; l++) {
8             double blj = B[l+j*k];
9             for (i = 0; i < m; i++) {
10                 double ail = A[i+l*m];
11                 C[i+j*m] += ail*blj;
12             }
13         }
14     }
15     return 0;
16 }
```

3.1.2 OpenMP offloading を用いた行列積演算

次に OpenMP offloading を用いた行列積演算をソースコード 3.2 に示す. OpenMP と異なり, 2 つの for ループを結合する `collapse(2)` がディレクティブとして使用するの, CPU よりも圧倒的にコア数が多く, 計算速度が速くなった為である.

ソースコード 3.2 OpenMP offloading を用いた行列積演算プログラム

```
1 int matmul()
2 {
3     int i, j, l;
4     #pragma omp target map(to: A[:m*k], B[:k*n], C[:m*n]) map(from: C[:m
5         *n])
6     #pragma omp teams distribute parallel for collapse(2)
7     for (j = 0; j < n; j++) {
8         for (i = 0; i < m; i++) {
9             for (l = 0; l < k; l++) {
10                 double ail = A[i+l*m];
11                 double blj = B[l+j*k];
```

```

11             C[i+j*m] += ail*blj;
12         }
13     }
14 }
15     return 0;
16 }

```

3.1.3 OpenACC を用いた行列積演算

最後に OpenACC における行列積計算をソースコード 3.3 に示す.

ソースコード 3.3 OpenACC を用いた行列積演算プログラム

```

1 int matmul()
2 {
3     int i, j, l;
4     #pragma acc data copyin(A[0:m*k],B[0:k*n]),copy(C[0:m*n])
5     #pragma acc kernels
6     #pragma acc loop independent
7     for (j = 0; j < n; j++) {
8         #pragma acc loop independent
9         for (i = 0; i < m; i++) {
10            #pragma acc loop seq
11            for (l = 0; l < k; l++) {
12                double ail = A[i+l*m];
13                double blj = B[l+j*k];
14                C[i+j*m] += ail * blj;
15            }
16        }
17    }
18    return 0;
19 }

```

3.2 diffusion

diffusion プログラムとは, 2 次元グリッドにおいて, それぞれのグリッドの情報を 0~1.0 で格納し, 時間ステップごとに隣り合うグリッドと自身のグリッドの情報の平均値を, 自身のグリッドの情報として新たに格納するプログラムである. このプログラムでは $nx \times ny$ のグリッドを想定し, イテレーションを nt 回としており, グリッドの角や辺の情報は固定し, 境界における計算は行わない.

diffusion プログラムの初期条件をソースコード 3.4 に示す. ここでは初期条件としてグリッド上部の中心から半径 $(nx+ny)/8$ 以内に存在するグリッド格子を 1, それ以外のグリッド格子を 0 としている.

ソースコード 3.4 diffusion プログラムの初期条件

```
1 #define IDX(b,x,y) ((b)*(ny)*(nx) + (y)*(nx) + (x))
2 void init(long nx, long ny)
3 {
4     long x, y;
5     long cx = nx/2, cy = 0; /* center of ink */
6     long rad = (nx+ny)/8; /* radius of ink */
7     data = (float *)malloc(bufsize);
8
9     #pragma omp parallel for private(x)
10    for(y = 0; y < ny; y++) {
11        for(x = 0; x < nx; x++) {
12            float v = 0.0;
13            if (((x-cx)*(x-cx)+(y-cy)*(y-cy)) < rad*rad) {
14                v = 1.0;
15            }
16            data[IDX(0,x,y)] = v;
17            data[IDX(1,x,y)] = v;
18        }
19    }
20    return;
21 }
```

3.2.1 OpenMP と OpenMP offloading を用いた diffusion プログラム

OpenMP と OpenMP offloading を用いたプログラムをソースコード 3.5 に示す. ただし, OpenMP offloading を使用する場合は, OMP_OFFLOAD が定義されているものとする.

ソースコード 3.5 OpenMP と OpenMP offloading を用いた diffusion プログラム

```
1 void calc(long nx, long ny, long nt){
2     long t, x, y;
3
4     #ifdef OMP_OFFLOAD
5     #pragma omp target data map(tofrom: data[0:2*nx*ny])
6     #endif
7     {
8         for (t = 0; t < nt; t++) {
9             long from = t%2;
10            long to = (t+1)%2;
11
12            #ifdef OMP_OFFLOAD
13            #pragma omp target teams distribute parallel for collapse(2)
14            #else
15            #pragma omp parallel for private(x)
16            #endif
17            for (y = 1; y < ny-1; y++) {
18                for (x = 1; x < nx-1; x++) {
19                    data[IDX(to,x,y)] = 0.2 * (data[IDX(from,x,y)]
20                                                + data[IDX(from,x-1,y)]
21                                                + data[IDX(from,x+1,y)]
22                                                + data[IDX(from,x,y-1)]
23                                                + data[IDX(from,x,y+1)]);
24                }
25            }
26        }
27    }
28    return;
29 }
```

3.2.2 OpenACC を用いた diffusion プログラム

次に, OpenACC を用いたプログラムをソースコード 3.6 に示す.

ソースコード 3.6 OpenMP と OpenMP offloading を用いた diffusion プログラム

```
1 void calc(long nx, long ny, long nt)
2 {
3     long t, x, y;
4     #pragma acc data copy(data[0:2*nx*ny])
5     {
6         for (t = 0; t < nt; t++) {
7             long from = t%2;
8             long to = (t+1)%2;
9
10            #pragma acc kernels
11            #pragma acc loop independent
12            for (y = 1; y < ny-1; y++) {
13                #pragma acc loop independent
14                for (x = 1; x < nx-1; x++) {
15                    data[IDX(to,x,y)] = 0.2 * (data[IDX(from,x,y)]
16                                                + data[IDX(from,x-1,y)]
17                                                + data[IDX(from,x+1,y)]
18                                                + data[IDX(from,x,y-1)]
19                                                + data[IDX(from,x,y+1)]);
20                }
21            }
22        }
23    }
24    return;
25 }
```

3.3 hotspot プログラム

hotspot プログラムとは並列コンピューティングの性能を評価することを目的とした Rodinia ベンチマーク [6] に含まれるプログラムである。主に CPU や GPU を用いた異種コンピューティング環境向けの性能評価を行う Rodinia ベンチマークには、画像処理や科学技術計算、医療シミュレーションや機械学習などといった多種多様なプログラムが、それぞれ OpenMP, CUDA, OpenCL など様々な言語に対応した形で含まれている。その中でも、熱伝導シミュレーションを行う hotspot プログラムを用いて性能評価を行う。このプログラムは特に集積回路の熱分布を解析する目的で設計されている。

3.3.1 OpenMP と OpenMP offloading における hotspot プログラム

hotspot プログラムでは、チップの熱抵抗や熱容量、周囲温度、さらには各セルの消費電力などによって、2次元グリッドにおける温度計算を時間ステップごとに行う。しかし、元のプログラムでは OpenMP offloading で実装されたプログラムがない為、OpenMP で実装されたプログラムに対し、指示文の改変 (6~10 行目) を行った。

具体的なプログラムを紹介する前に、各変数についての説明を行う。result は温度計算の結果を格納する配列、temp は現在の各格子上での温度を格納する配列であり、どちらもサイズは row 行 × col 列で設定されている。また、Cap_1, Rx_1, Ry_1, Rz_1, step は熱容量、それぞれ x 方向の熱抵抗、y 方向の熱抵抗、z 方向の熱抵抗、ステップの回数を表す。power は各セルで発生する消費電力を表し、amb_temp は周囲の温度を表す。

プログラムの計算の概要について説明をすると、まず最初に指定されたブロックサイズでブロッキングを行い、頂点と辺が含まれたブロックと内部のブロックの計算を並列化によって同時に行い、結果を result に代入する。ブロックの行と列の長さはそれぞれ、BLOCK_SIZE_R, BLOCK_SIZE_C で表される。ソースコード 3.7 に OpenMP と OpenMP offloading を用いた hotspot プログラムを示す。ただし、OpenMP offloading を用いる場合は OMP_OFFLOAD が定義されているものとする。

ソースコード 3.7 OpenMP と OpenMP offloading を用いた hotspot プログラム

```
1 void single_iteration(FLOAT *result, FLOAT *temp, FLOAT *power, int
    row, int col, FLOAT Cap_1, FLOAT Rx_1, FLOAT Ry_1, FLOAT Rz_1,
    FLOAT step){
2     FLOAT delta;
3     int r, c, chunk;
4     int num_chunk = row*col / (BLOCK_SIZE_R * BLOCK_SIZE_C);
5     int chunks_in_row = col/BLOCK_SIZE_C;
6 #ifdef OMP_OFFLOAD
7 #pragma omp target teams distribute parallel for shared(power, temp,
    result) private(chunk, r, c, delta) firstprivate(row, col,
    num_chunk, chunks_in_row) schedule(static)
8 #else
9 #pragma omp parallel for shared(power, temp, result) private(chunk, r,
    c, delta) firstprivate(row, col, num_chunk, chunks_in_row)
    schedule(static)
10 #endif
11     for ( chunk = 0; chunk < num_chunk; ++chunk ){
12         int r_start = BLOCK_SIZE_R*(chunk/chunks_in_row);
13         int c_start = BLOCK_SIZE_C*(chunk%chunks_in_row);
14
15         // グリッドの角と辺を含むブロックの計算
16
17         // . . .
18
19         // 内部ブロックの計算
20         for ( r = r_start; r < r_start + BLOCK_SIZE_R; ++r ) {
21 #pragma omp simd
22             for ( c = c_start; c < c_start + BLOCK_SIZE_C; ++c ) {
23                 /* Update Temperatures */
24                 result[r*col+c] =temp[r*col+c]+
25                     ( Cap_1 * (power[r*col+c] +
26                     (temp[(r+1)*col+c] + temp[(r-1)*col+c]
27                     - 2.f*temp[r*col+c]) * Ry_1 +
28                     (temp[r*col+c+1] + temp[r*col+c-1]
29                     - 2.f*temp[r*col+c]) * Rx_1 +
30                     (amb_temp - temp[r*col+c]) * Rz_1));
```

```

31         }
32     }
33 }
34 }

```

3.4 ブロッキング機能を取り除いた hotspot プログラム

hotspot プログラムを改変し、ブロッキング機能を取り除いたプログラムをソースコード 3.8 に示す。ブロッキングした場合と大きく違うのは、頂点と辺、それから内部のグリッドにおける計算を順番に行っていることである。

ソースコード 3.8 ブロッキング機能を取り除いた hotspot プログラム

```

1 void single_iteration(FLOAT *result, FLOAT *temp, FLOAT *power, int
    row, int col,
2
3     FLOAT Cap_1, FLOAT Rx_1, FLOAT Ry_1, FLOAT Rz_1,
4     FLOAT step){
5
6     int r, c;
7
8     // 四つの角の計算
9
10    // ...
11
12    #ifdef OMP_OFFLOAD
13    #pragma omp target teams distribute parallel for schedule(static)
14    #else
15    #pragma omp parallel for schedule(static)
16    #endif
17    for(c=1; c < col-1; ++c){
18
19        // 上辺と下辺の計算
20
21    }
22    #ifdef OMP_OFFLOAD
23    #pragma omp target teams distribute parallel for schedule(static)
24    #else
25    #pragma omp parallel for schedule(static)

```

```

24 #endif
25     for(r=1; r < row-1; ++r){
26
27         // 左辺と右辺の計算
28
29     }
30 // 内部グリッドの計算
31 #ifdef OMP_OFFLOAD
32 #pragma omp target teams distribute parallel for collapse(2) schedule(
    static)
33 #else
34 #pragma omp parallel for collapse(2) schedule(static)
35 #endif
36     for ( r = 1 ; r < row-1; ++r ) {
37         for ( c = 1; c < col-1; ++c ) {
38             /* Update Temperatures */
39             result[r*col+c] =temp[r*col+c]+
40                 ( Cap_1 * (power[r*col+c] +
41                     (temp[(r+1)*col+c] + temp[(r-1)*col+c] - 2.f*temp[r*
42                         col+c]) * Ry_1 +
43                     (temp[r*col+c+1] + temp[r*col+c-1] - 2.f*temp[r*col+c
44                         ]) * Rx_1 +
45                     (amb_temp - temp[r*col+c]) * Rz_1));
46         }
47     }

```

第 4 章

実験と評価

前章のプログラムを用いて研究室マシン 3 台で実験を行った。第 1 節「実行環境」では使用する研究室マシンとコンパイラの紹介を行い、第 2 節「各プログラムの実験方法」では前章で紹介した 3 つのプログラムの実験方法を紹介する。また、第 3 節「OpenMP offloading を用いた性能」と第 4 節「OpenMP と OpenACC との比較実験」では行列積演算, diffusion プログラムを用いて、第 5 節「性能ポータビリティ」では行列積演算, hotspot プログラムを用いて実験と評価を行った。

4.1 実行環境

本研究で使用する研究室マシンのスペックと各並列化手法におけるコンパイラについて説明する。

4.1.1 研究室マシン

本研究では 3 台の研究室マシンを用いて実験を行った。マシン 1 では AMD GPU, マシン 2 とマシン 3 ではスペックの異なる NVIDIA GPU が使用されており、3 台で使用されている GPU のメーカーやスペックが異なることに注意が必要である。さらに詳しく、マシン 1, マシン 2, マシン 3 のスペックをそれぞれ表 4.1, 表 4.2, 表 4.3 に示す。

表 4.1 マシン 1 の構成

CPU	Intel(R) Xeon(R) Silver 4215 2.5GHz × 2CPU
コア数/スレッド	8 コア / 16 スレッド × 2 CPU
メモリ容量	376 GiB
GPU	AMD Instinct MI210 × 1 GPU
コンピュータユニット (CU)	104 個
ストリームプロセッサ数	64 基 / CU
SM	104 個
L1 キャッシュ容量	16 KiB / SM
L2 キャッシュ容量	16 MiB
GPU メモリ容量	64 GiB HBM2e / GPU
FP32	22.6 TFlops
FP 64	22.6 TFlops
メモリ帯域幅	1.6 TB/s

4.1.2 コンパイラ

次に OpenMP, OpenMP offloading, OpenACC で実装されたプログラムをコンパイルする際に使用するコンパイラを表 4.4 に示す. 具体的なコマンドとしては OpenMP では `gcc -O3 -g -fopenmp`, OpenMP offloading ではマシン 1 の場合に `amdclang -O3 -g -fopenmp -offload-arch=gfx90a -DOMP_OFFLOAD` を, マシン 2, マシン 3 の場合に `nvc -O3 -g -mp=gpu -DOMP_OFFLOAD` を使用する. また, OpenACC では `nvc -O3 -g -acc` をコマンドとして使用する.

表 4.2 マシン 2 の構成

CPU	AMD EPYC 7702 64-Core Processor × 2 CPU
コア数/スレッド	64 コア / 64 スレッド × 2 CPU
メモリ容量	503GiB
GPU	NVIDIA A100 80GB PCIe × GPU
CUDA コア数	6912 個
Tensor コア数	432 個
SM	108 個
L1 キャッシュ容量	192 KiB / SM
L2 キャッシュ容量	40 MiB
GPU メモリ容量	80 GiB HBM2e / GPU
FP32	19.5 TFlops
FP64	9.7 TFlops
メモリ帯域幅	1935 GB/s

表 4.3 マシン 3 構成

CPU	AMD EPYC 7773X 64-Core Processor × 2 CPU
コア数/スレッド	64 コア / 64 スレッド × 2 CPU
メモリ容量	251GiB
GPU	NVIDIA H100 PCIe × 1 GPU
CUDA コア数	14592 個
Tensor コア数	456 個
SM	114 個
L1 キャッシュ容量	256 KiB / SM
L2 キャッシュ容量	50 MiB
GPU メモリ容量	80 GiB HBM2e / GPU
FP32	51 TFlops
FP 64	26 TFlops
メモリ帯域幅	2.0 TB/s

表 4.4 使用したコンパイラの一覧

	OpenMP	OpenMP offloading	OpenACC
マシン 1	gcc 8.5.0	clang 17.0.6	-
マシン 2	gcc 8.5.0	NVIDIA HPC SDK 21.7	NVIDIA HPC SDK 21.7
マシン 3	gcc 8.5.0	NVIDIA HPC SDK 21.7	NVIDIA HPC SDK 21.7

4.2 各プログラムの実験方法

各研究室マシンで行う、それぞれのプログラムの実験方法を説明する。特に第3節「OpenMP offloading を用いた性能」と「OpenMP と OpenACC との比較実験」には実験方法で明確な違いがあるので注意が必要である。第3節では、データ転送時間を測定しつつ、転送コストを除いた実行速度、すなわち GPU での実行速度を測定している。それに対し第4節では、CPU 並列化手法である OpenMP との比較も行う為、OpenMP offloading や OpenACC を用いる場合は、GPU での実行速度に加えて、転送コストを含めた実行速度、すなわち研究室マシンにおける実行速度も測定している。なお、いずれの場合も GPU の初期化コストは含まれない。

4.2.1 行列積演算

行列積演算プログラムでは行列サイズを変えて、11 回連続で計算を行い、最初の 1 回を除いた 10 回の計算速度の平均を算出した。最初の 1 回を計算に含めないのは、1 回目の計算で GPU での初期化が行われてしまう為である。また、本研究では同じサイズの正方行列同士の行列積演算を行う為、正方行列の 1 辺の長さを行列サイズと呼ぶことにする。

4.2.2 diffusion

diffusion プログラムでは、イテレーションを 1000 回として、グリッドサイズを変えながら、その計算速度を 1 回測定した。なお、GPU での初期化コストは省く為、測定の前にイテレーション 1 回で計算を行った。

4.2.3 hotspot

hotspot プログラムでは、グリッドサイズを 1024×1024 、イテレーションを 10000 回として、ブロックサイズを変えながら計算速度を 1 回測定した。なお、GPU での初期化コストは省く為、測定の前にイテレーション 1 回で計算を行った。

4.3 OpenMP offloading を用いた性能

各研究室マシンにおいて、OpenMP offloading で実装された行列積演算と diffusion プログラムを実行した。以下、GPU の実行速度とは、GPU での初期化コスト及び CPU、GPU 間の転送コストを除いた実行速度を意味する。

4.3.1 各 GPU 上での実行速度

まず、各研究室マシンで OpenMP offloading で実装された行列積演算プログラムを行列サイズを変えながら実行し、GPU の実行速度を測定した。各 GPU による結果を図 4.1 に示す。どの GPU も行列サイズ 1000~2000 で実行速度が最も速くなり、行列サイズが大きくなるにつれ速度が減少し、やがてほとんど一定となった。

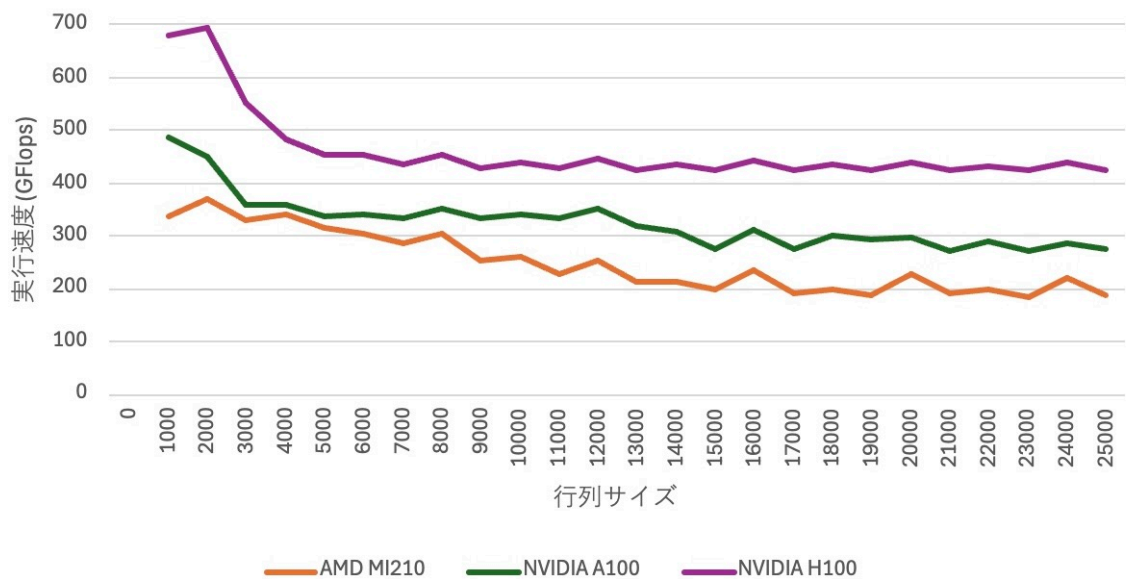


図 4.1 各 GPU による行列積演算の実行速度

次に、各研究室マシンで OpenMP offloading で実装された diffusion プログラムをグリッドサイズを変えながら実行し、GPU の実行速度を測定した。各 GPU による結果を図 4.2 に示す。グリッドサイズを大きくしていくと、NVIDIA A100, NVIDIA H100 は

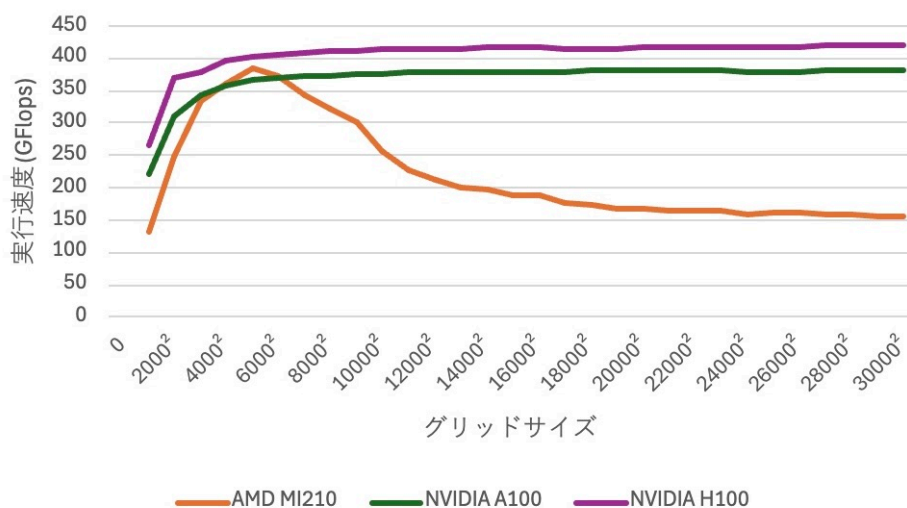


図 4.2 各 GPU による diffusion の実行速度

実行速度が増加していき、やがてほとんど一定になるのに対して、AMD MI210 は実行速度がピークに達すると、その後大きく低下していき、やがてほとんど一定となった。AMD MI210 が他の GPU よりキャッシュサイズが小さく、キャッシュミスを起こしたためだと考えられる。行列積演算の場合と異なる結果になったことを踏まえて、この推測が正しいのか今後の研究で検証が必要である。

4.3.2 各研究室マシンの転送時間

OpenMP offloading などの GPU 並列化手法を使用する際に、GPU 初期化コストの他に、CPU、GPU 間のデータ転送にかかる時間が存在する。これらを行列積演算、diffusion プログラムを用いて研究室マシン 3 台で測定し、転送速度を算出した。まず、行列積演算プログラムの各研究室マシンにおける転送速度を図 4.3 に示す。なお、転送時間は「データ転送を含めた実行時間 (s)-GPU での実行時間 (s)」として求め、転送速度は「データ転送量 (B)÷ データ転送時間 (s)」として算出した。

次に、diffusion プログラムの各研究室マシンにおける転送速度を図 4.4 に示す。全体として、行列積演算より diffusion での実行速度が遅いことが分かる。また、行列積演算ではマシン 2、マシン 3 とともに転送速度が行列サイズに依存せず安定しているが、diffusion プ

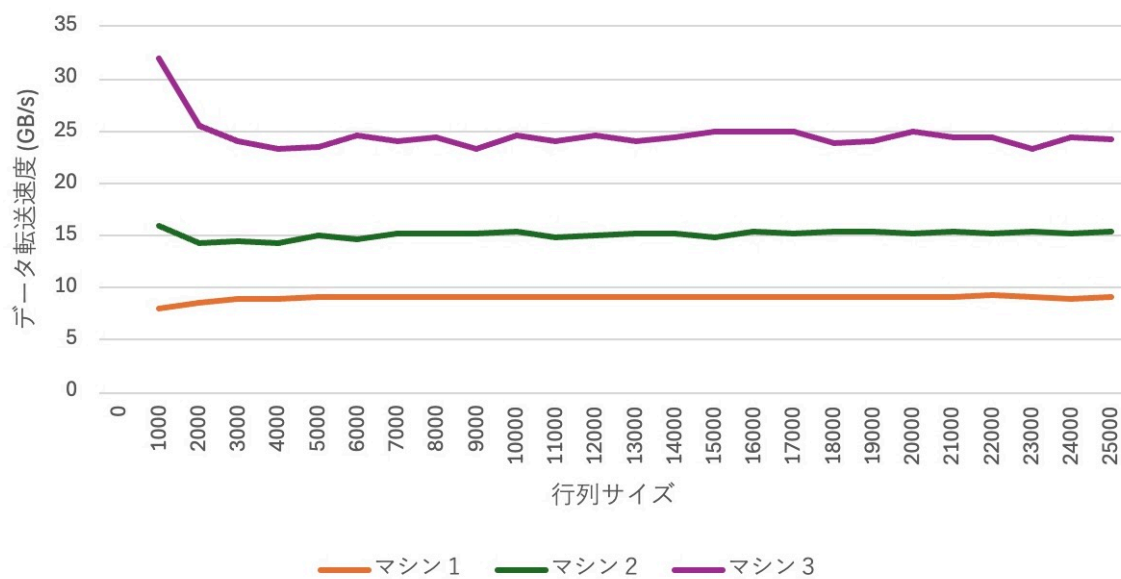


図 4.3 行列積演算プログラムでのデータ転送速度

ログラムでは両者とも転送速度が不安定であることが読み取れる。プログラムによって転送速度は変わってしまった原因や、マシン 2 とマシン 3 で転送速度が不安定になった原因について調べるのが今後の課題である。

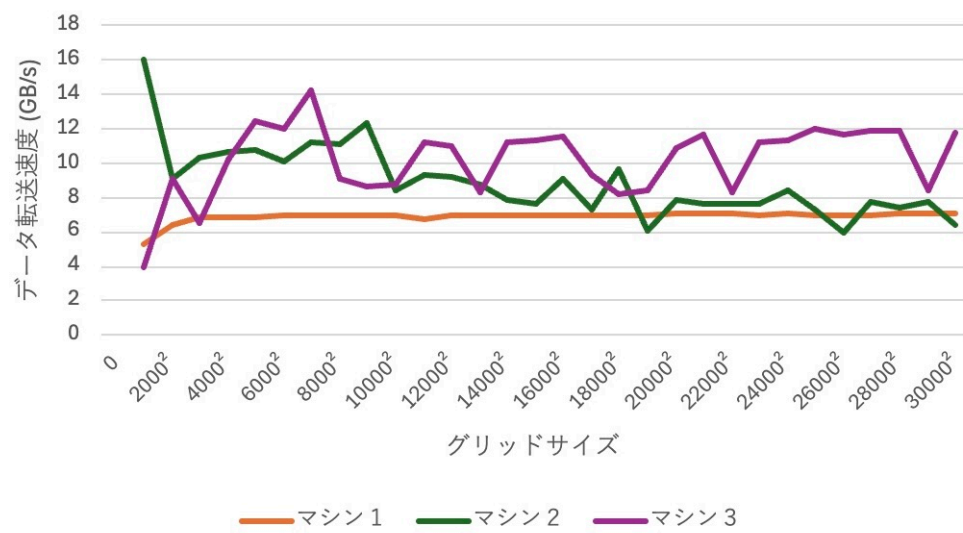


図 4.4 diffusion プログラムでの転送速度

4.4 OpenMP と OpenACC との比較実験

CPU, GPU 並列化手法における性能の違いや, OpenMP offloading の GPU 並列化手法としての性能を測るため, OpenMP offloading で実装したプログラムと, OpenMP, OpenACC で実装したプログラムの比較を研究室マシン 3 台を用いて行った. なお, OpenACC は NVIDIA GPU 向けの並列化手法である為, マシン 1 では実験対象外とする. 行列積演算における各並列化手法の実行速度を図 4.5, 4.6, 4.7, 4.8, 4.9 に示す.

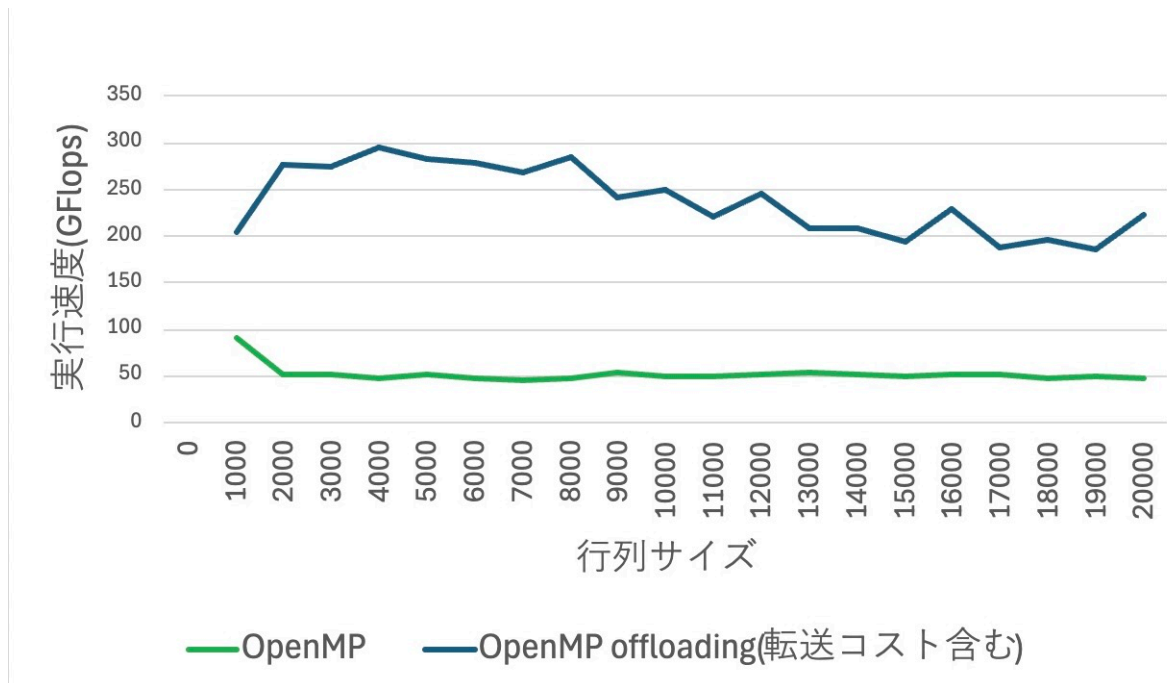


図 4.5 マシン 1 OpenMP との行列積演算速度比較

実験を通して, CPU の性能が高いマシン 2 とマシン 3 では, 行列積演算において, サイズが小さい時の実行速度は OpenMP を用いた場合が最も速いことが分かった. これは, GPU 並列化手法での転送コストによる影響だとグラフから読み取れる. しかし, サイズが大きくなると OpenMP での性能は著しく低下し, GPU 並列化手法を用いた場合の方が優位な結果となった. これは CPU はコア数が 32 から 128 個であるのに対し, GPU のコア数が数千から数万個あり, GPU の方が大規模計算に向いている為だと考えられる. 一方, OpenMP offloading を用いた場合と OpenACC を用いた場合の実行速度による差は

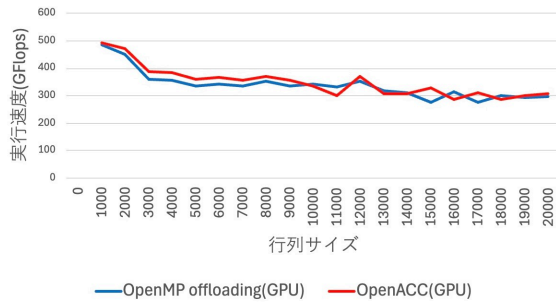


図 4.6 マシン 2 OpenMP との行列積演算速度比較

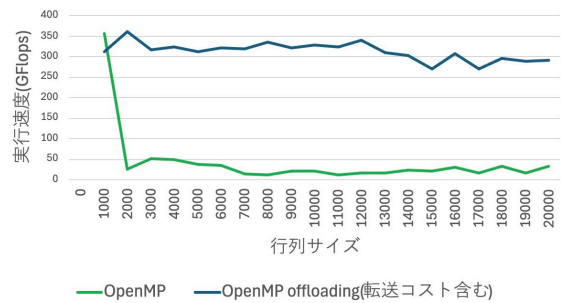


図 4.7 マシン 2 OpenACC との行列積演算速度比較

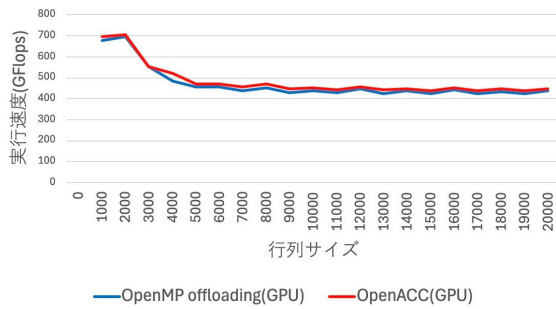


図 4.8 マシン 3 OpenMP との行列積演算速度比較

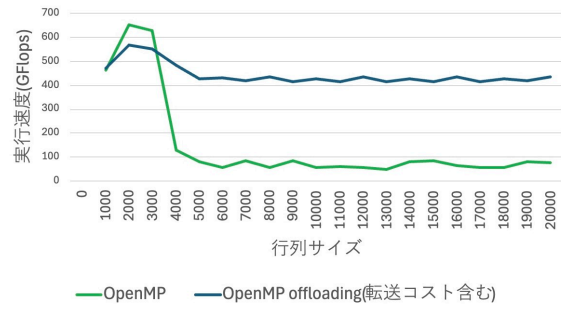


図 4.9 マシン 3 OpenACC との行列積演算速度比較

ほとんど見られなかった。

次に, diffusion プログラムにおける各並列化手法の実行速度を図 4.12, 4.13, 4.14 に示す。

memkf01 の実験からは OpenMP より OpenMP offloading を用いた場合の方が実行速度が常に優位であることが分かる。一方, memkf03 と lupus を用いた実験では, CPU の性能の高さ故に, 特定のグリッドサイズにおいて, OpenMP が OpenMP offloading に近い性能を発揮することが分かった。しかし, 全体として GPU 並列化手法を用いた場合の方が実行速度が速いことが分かった。また, 行列積演算プログラムではそれぞれ OpenMP offloading, OpenACC を用いた場合に大きな性能の違いは見られなかったが, diffusion プログラムでは全体的に, OpenMP offloading を用いた場合は OpenACC を用いた場合の 0.7 倍から 0.8 倍の実行速度となった。

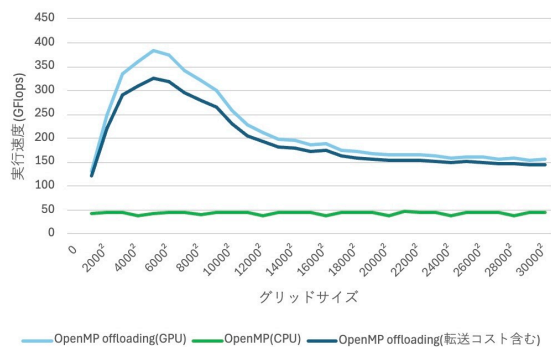


図 4.10 マシン 1 における diffusion の実行速度

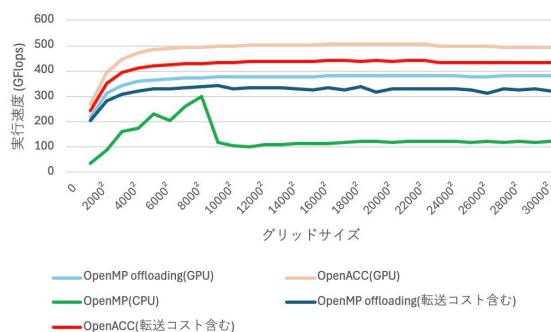


図 4.11 マシン 2 における diffusion の実行速度

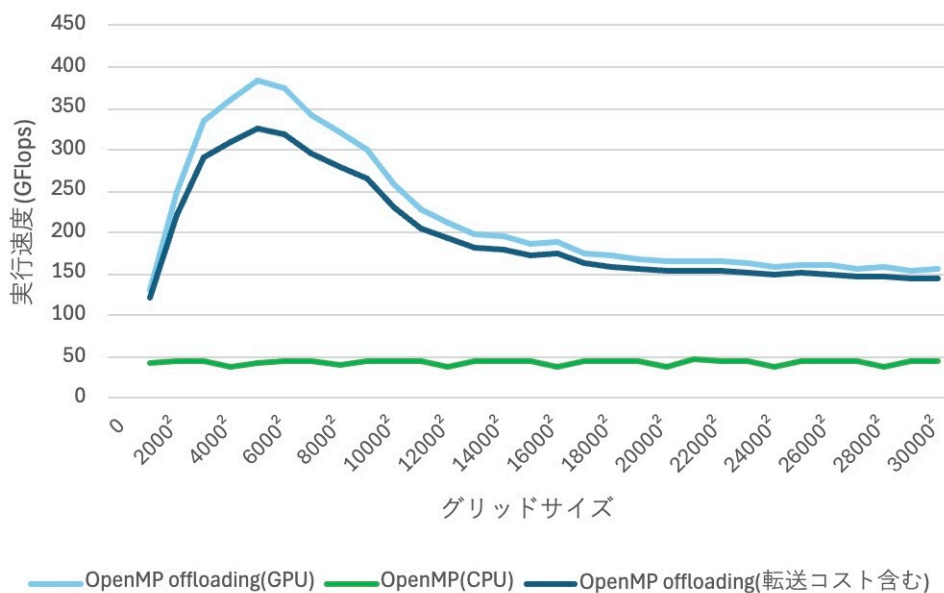


図 4.12 マシン 1 における diffusion の実行速度

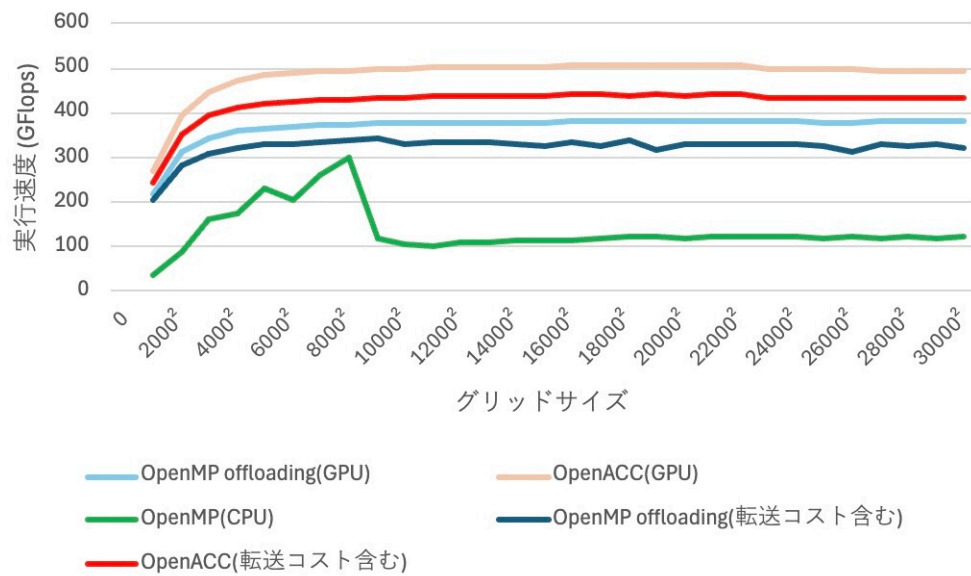


図 4.13 マシン 2 における diffusion の実行速度

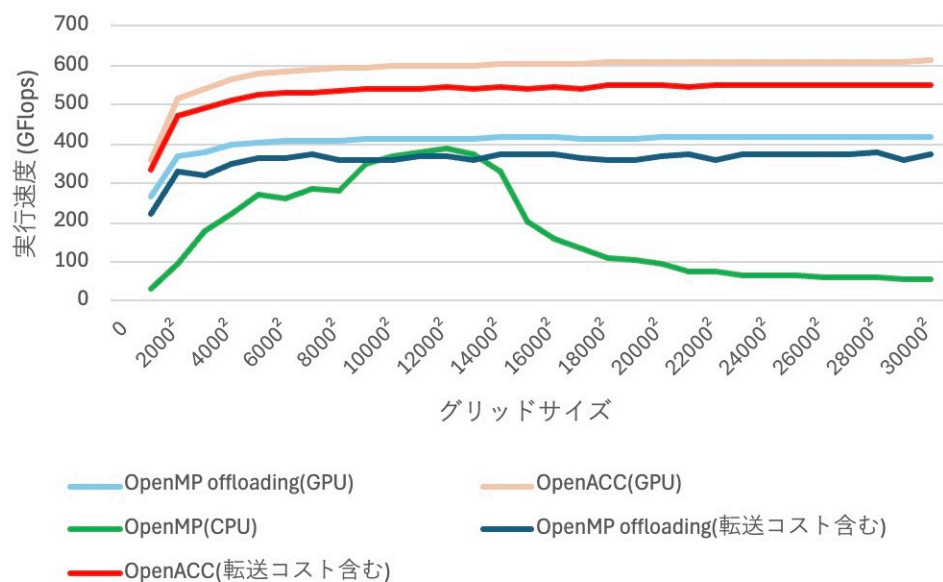


図 4.14 マシン 3 における diffusion の実行速度

4.5 性能ポータビリティ

性能ポータビリティとは、ソフトウェアを異なる利用環境に移動する時に安定した性能が出せる能力のことを指す。本研究では、同一のプログラムを OpenMP, OpenMP offloading, OpenACC で実装した場合に性能ポータビリティが保たれているかを評価する。

4.5.1 行列積演算における性能ポータビリティ

まず、行列積演算の 3 重ループのうち内側 2 つのループの順番が異なるプログラムを用意し、2 つのプログラムが各並列化手法で性能が保たれているかを実験で確認する。

実験の結果を図 4.15, 4.16, 4.17 に示す。なお、第 3 章で紹介したコードは行内の書き換えは行わず、順番を入れ替えるだけとする。

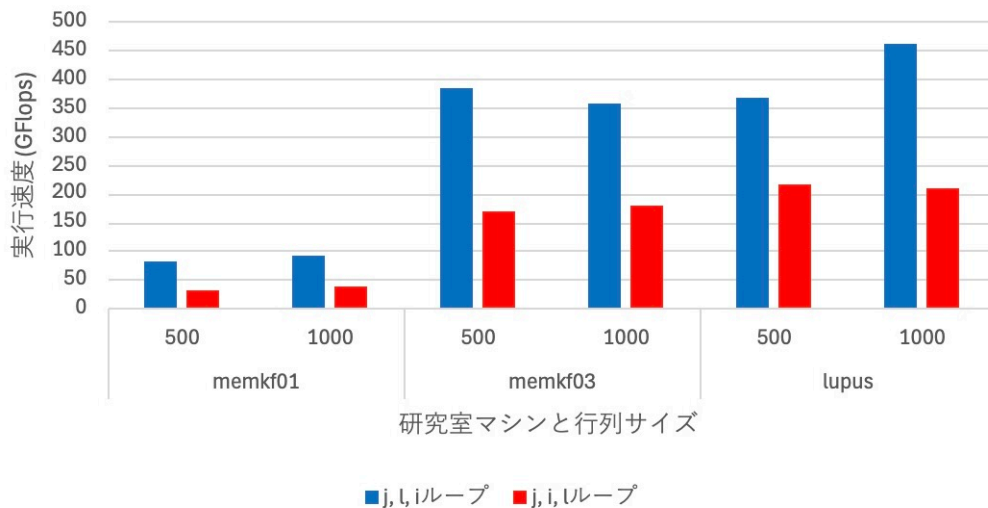


図 4.15 異なる for ループでの OpenMP 実行速度

図 4.15 から、OpenMP を用いた行列積演算プログラムでは j, i, l ループより j, l, i ループのプログラムの方が速いことが分かる。これは j, l, i ループの方が、一番内側のループでのキャッシュミスが起こりにくい為であると推測される。一方、図 4.16 から OpenMP

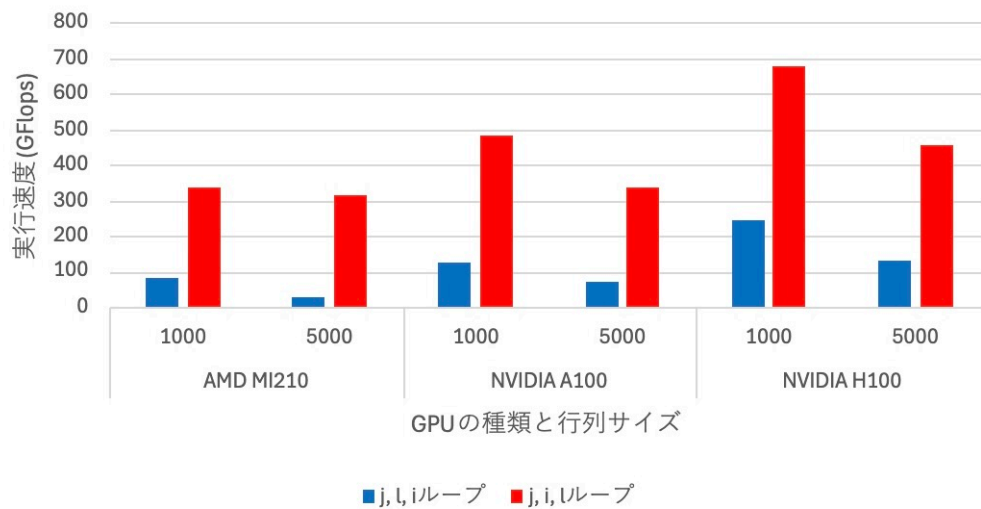


図 4.16 異なる for ループでの OpenMP offloading 実行速度

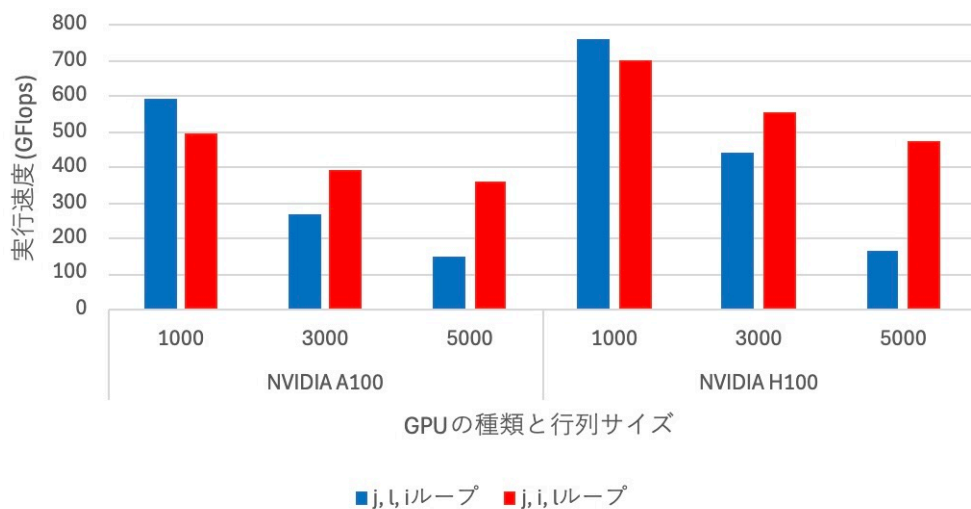


図 4.17 異なる for ループでの OpenACC 実行速度

offloading を用いた場合は j, i, l ループの方が速いことが分かる。これは GPU のコア数の多さを活かして、j, i ループを結合して並列化した為だと考えられる。すなわち、一番内側のループでのキャッシュミスの起こりにくさよりも CPU, GPU におけるコア数が与える影響の方が大きいと言える。

また, OpenACC を用いた場合には, 行列サイズが 1000 の時は j, l, i ループの方が, 行列サイズが 3000, 5000 の時は j, i, l ループの方が実行速度が速くなった. j, l, i ループでは, i ループでの計算が全て終わるまで l が増えないので, 行列サイズが大きくなるにつれ, キャッシュミスが起こりにくい恩恵よりも, その影響が大きくなる為だと考えられる. しかし, OpenMP offloading で同様の結果とならなかったのは OpenACC との文法の制限の違いに因る. つまり, OpenMP offloading で複数のループを並列化するにはそれらのループは連続していないといけなのに対し, OpenACC では「`#pragma acc loop independent`」が連続していないループでも使用できる. それにより, 配列 C に書き込む際衝突が起きない j, i ループの並列化を j, l, i ループでも可能にしたということである.

4.5.2 hotspot プログラムにおける性能ポータビリティ

ブロッキングが施されている元の hotspot プログラムと, それを元に独自で作成したブロッキング機能のない hotspot プログラムを OpenMP と OpenMP offloading を用いて研究室マシン 3 台で実験した. ブロッキングが施されている hotspot プログラムに対してはブロックサイズを変えて実験を行い, OpenMP, OpenMP offloading それぞれを用いた場合, どのブロックサイズが適切であるか, またブロッキング機能を無くした場合の方が早いのかを実験を通して調べた. OpenMP による実験結果を図 4.18 に, OpenMP offloading による実験結果を図 4.19 に示す.

実験を通して, OpenMP を用いた場合ではマシンによって適切なブロックサイズが異なるのに対し, OpenMP offloading を用いた場合ではどの GPU においてもブロックサイズ 2×2 でブロッキングをした場合が最も実行速度が速くなることが分かった. また, ブロックサイズが大きくなるにつれ, OpenMP を用いた場合よりも実行速度が大幅に遅くなった. これらの原因として, 沢山ある GPU のコアを十分に使いきれていないことが考えられる. しかし, ブロッキング機能を無くした場合の方が, ブロックサイズ $2 \times 2 \sim 4 \times 4$ でブロッキングした場合よりも実行速度が遅くなってしまった原因は分からないので, これからの研究課題である.

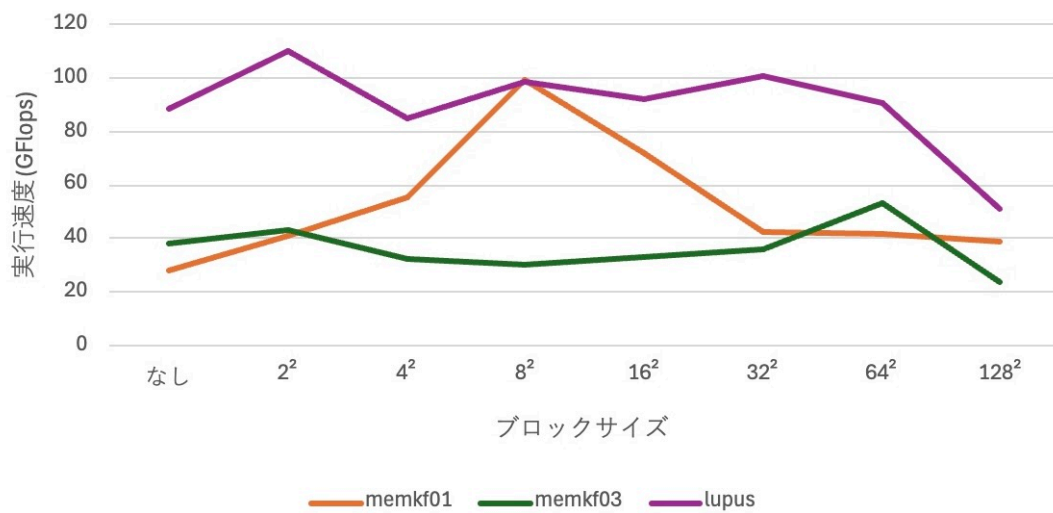


図 4.18 OpenMP での hotspot 実行速度

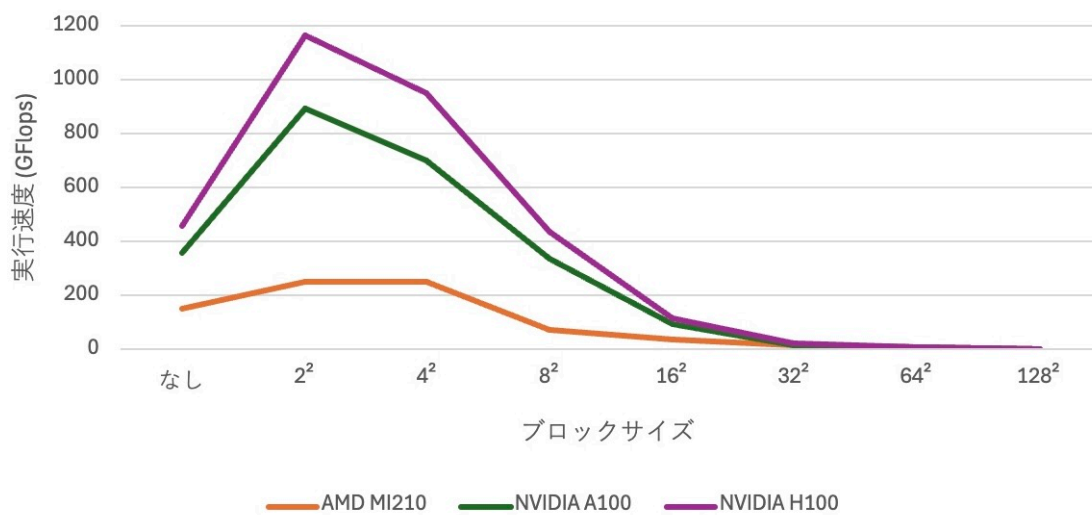


図 4.19 OpenMP offloading での hotspot 実行速度

第 5 章

関連研究

5.1 プログラミングフレームワーク

異なるハードウェアで動作するソフトウェアを構築するために、プログラミングフレームワークを作成した研究があるので紹介する。瓜生らの研究 [7] では、ステンシル計算という diffusion と似たプログラムにおいて、プログラミングコストの高い時間ブロッキングのフレームワークを実装し、そのフレームワークが OpenMP で実装したプログラム、さらには OpenMP を用いて時間ブロッキングしたプログラムと比べてどのような性能を出すか実験した。この研究では、配列サイズによって適切なブロックサイズが異なるという実験結果が得られたが、本研究で紹介した hotspot においてもグリッドサイズを変更すれば、CPU、GPU どちらにおいても適切なブロックサイズは変わってくる可能性が高いので、引き続きブロッキングに関する調査を進める必要がある。

第 6 章

結論

6.1 まとめ

OpenMP offloading を使用すると、行列積演算においてはキャッシュサイズの影響かもしれないが、AMD GPU での実行速度が極端に下がってしまうことや、diffusion プログラムにおいては NVIDIA GPU の転送速度が不安定であることなど、異種 GPU 間における差異も見られた。OpenMP と OpenACC との比較実験を通しては、GPU 並列化手法は転送コストがかかるという点で CPU に対して不利であるが、CPU に比べて圧倒的なコア数を持つ為、大規模計算には向いていることが分かった。また、OpenMP offloading は AMD GPU, NVIDIA GPU 上で利用できるという長所があるが、本研究のプログラムにおいて OpenACC と同等かそれ以下の実行速度であることが分かった。さらに、CPU, GPU 間では性能ポータビリティが失われるケースがあること、1つの並列化手法においてもサイズによって適切なプログラムが異なること、GPU では小規模計算におけるブロッキングはブロックサイズが小さい方が好ましいことなどが分かった。

6.2 今後の課題

キャッシュサイズがほとんど変わらない AMD GPU と NVIDIA GPU において OpenMP offloading を用いたプログラムを実行し、差異が生じるのかについて調査する必要がある。また、デバイス間の転送速度が安定しない理由についてデータの転送量を変えたり、様々なプログラムとデバイスを用いたりして調査したい。さらに様々なプログラム

や実行環境において、性能ポータビリティが失われてしまうケースを見つけ、原因についての調査を行いたい。また、hotspot に限らず、様々なプログラムにおいて適切なブロックサイズとその理由について CPU や GPU のコア数の観点から検証したい。そして、これらの調査結果を活かして、あらゆるプログラムで最適な並列化手法を選べるようになりたい。

謝辞

本論文を執筆するにあたり多くの助言と指針を示してくださいました遠藤先生に心から感謝致します。

また、研究方針などについて多くの助言をいただいた遠藤研究室の細木隆豊氏に感謝いたします。

最後に、本研究に取り組むにあたって、様々なご支援を頂きました遠藤研究室の皆様に感謝致します。

参考文献

- [1] 大島聡史. これからの並列計算のための GPGPU 連載講座 (I)GPU と GPGPU の歴史と特徴. <https://www.cc.u-tokyo.ac.jp/public/VOL12/No1/201001gpgpu.pdf>, 2010.
- [2] Leonardo Dagum and Ramesh Menon. OpenMP: an industry standard API for shared-memory programming. *IEEE computational science and engineering*, Vol. 5, No. 1, pp. 46–55, 1998.
- [3] OpenACC プログラミング by PGI – 2 章 OpenMP と OpenACC –. <https://hpcworld.jp/archive/SPG/Pgi/OpenACC/002.html>.
- [4] 河合 直聡, 三木 洋平, 星野 哲也, 塙 敏博, 中島 研吾. OpenMP を用いた GPU オフローディングの有効性の評価. 情報処理学会研究報告, Vol. 2022-HPC-183, No. 18, 2022 年.
- [5] The Top500 List. <https://www.top500.org/>.
- [6] OpenBenchMarking. <https://openbenchmarking.org/test/pts/rodinia>.
- [7] 瓜生 侑, 遠藤 敏夫. ステンシル計算の時間ブロッキングフレームワークの実装と評価. 情報処理学会研究報告, Vol. 2024-HPC-194, No. 3, 2024 年.